STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID: LRC – 26 – 0439	TEAM NAME: Srotas Pravaha

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity 2. Pumps – 3 Quantity 3. FT-01 Flow Transmitter – 01 Quantity 4. Ball valve – 3 Quantity 5. ON/OFF/XV Valve – 1 Quantity 6. 10-mm OD tubing/blue – 9 meters/1 meter 7. Microcontroller/PCB/ data-acquisition software 8. Required electrical/ electronic accessories	1. Custom PCB 2. ESP32 NodeMCU module 3. Pump Driver-1 4. Pump Driver-2 5. Pump Driver-3 6. 12 V DC power input (**SMPS**) 7. 12 VDC Solenoid-valve interface 8. Flow-sensor interface 9. Float-switch interface 10. ON/OFF-switch interface 11. Pump-driver PWM/enable interfaces 12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code**. The case-1 can be turned on using the available on/off Switch, to start the system.

CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

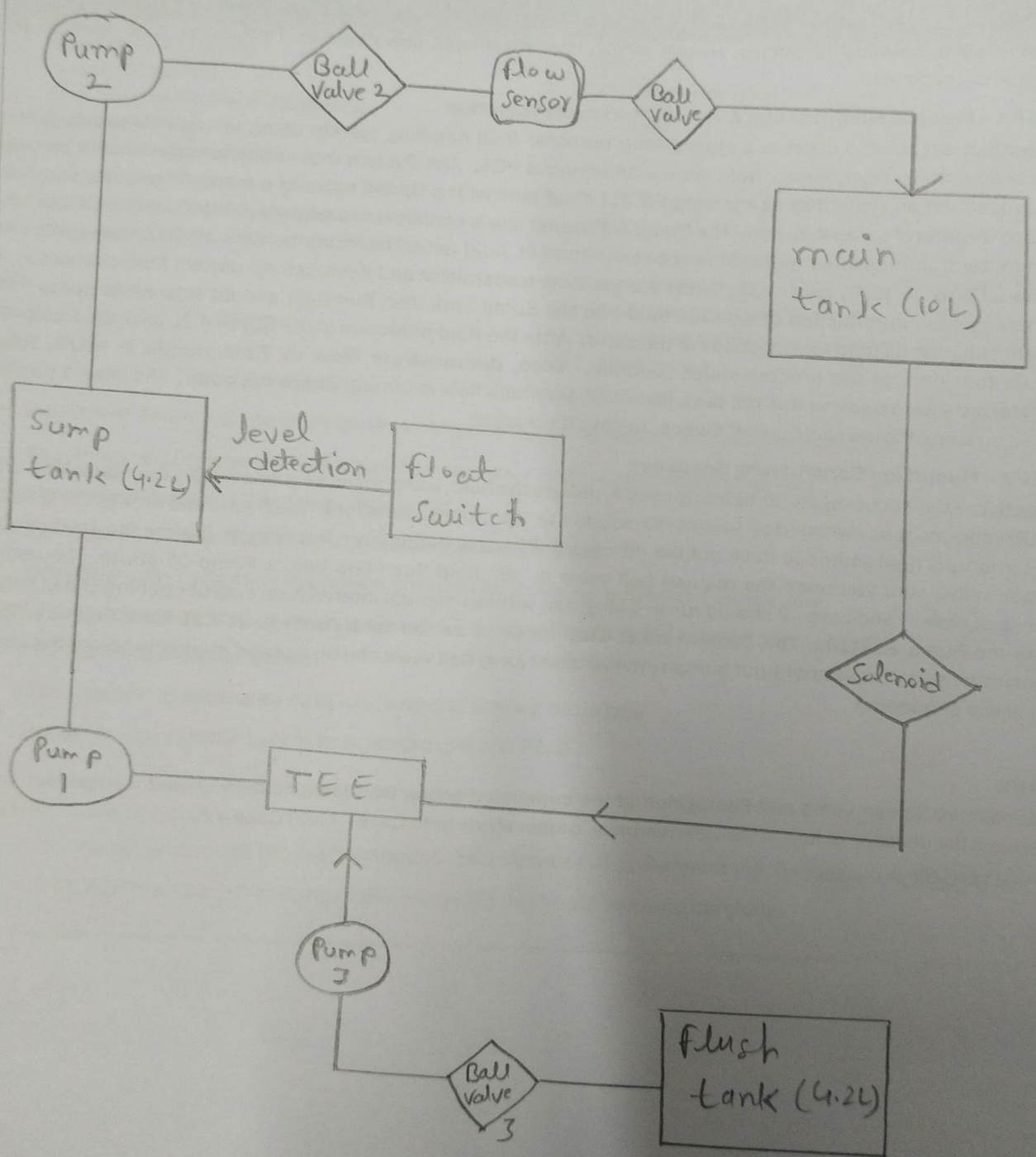
SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram

Team ID: LRC-26-0439 Team Name: Srotas Pravaha

P&ID SKETCH / WORKING SPACE



Pump maximum capacity is 2.805 L/min, not 6 L/min.

Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min) Compare the theoretical velocity with the actual operating behaviour.

→ ANALYSIS COMPARISON

the cross-sectional area is -

$$A = \frac{\pi D^2}{4} = \frac{\pi (0.008)^2}{4} = 5.0265 \times 10^{-5} m^2$$

the theoretical average velocity is -

$$v = \frac{Q}{A}$$

where Q must be expressed in m^3/s. to convert flow from L/min

$$Q\ m^3/s = \frac{Q\ L/min \times 10^{-3}}{60}$$

* 6L/min was not possible because pumps maximum capacity is [2.805 L/min]

• 1L/min →

$$V_{theoretical} = [0.332\ m/s]$$

$$V_{actual} = \frac{0.3504}{20} \times 60 = \frac{1.05 \times 10^{-3}}{60} \text{ m/s} = [0.345\ m/s]$$

$$\frac{}{5.0265 \times 10^{-5}}$$

• 3L/min ⇒ $V_{theoretical} = [0.995\ m/s]$

$$V_{actual} = \frac{2.805 \times 10^{-3}}{60} = [0.930\ m/s]$$

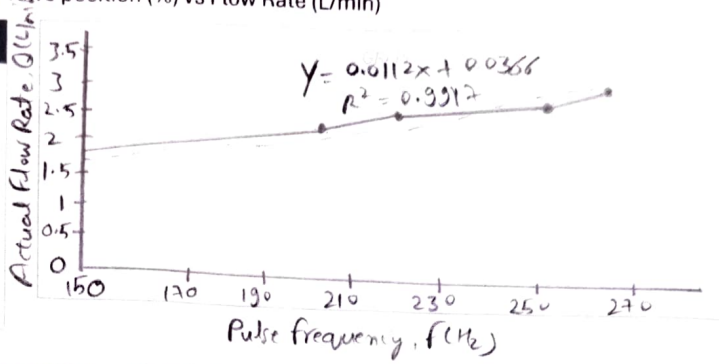
$$\frac{}{5.0265 \times 10^{-5}}$$

Actual behaviour may differ from theoretical prediction because of pump performance, hydraulic resistance, fitting valve characteristics and fluctuations in the system.

FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	2.460
50%	2.610
70%	2.700
100%	2.805



Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 = 0.9917

a = 0.0112

b = 0.0366

Maximum calibration error = ~~0.01275~~ 0.0625

Average calibration error = ~~0.00806~~ 0.0396

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	5086	20	254.30	935	2.805
2	4922	20	246.10	900	2.700
3	4441	20	222.05	870	2.610
4	4053	20	202.65	820	2.460
5	0	20	0	0	0

CALCULATIONS / OBSERVATIONS

Actual flow →

$$Q_{actual} = V_L / 20 \times 60$$

$$\text{Error}(\%) = \frac{|Q_{predicted} - Q_{actual}|}{Q_{actual}} \times 100$$

The R^2 value of approximately 0.966 indicates that the four calibration measurements follow a reasonably strong linear trend.

HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER correct formula is $[T_r = V/Q]$

1) Residence time physically indicates how long remains in specified tubing section before being displaced by incoming fluid.

2) Residence time doubles because the same flow rate must transport twice as much liquid volume.

3) Residence time is halved because fluid passes through the unchanged tubing volume twice as quickly.

4) Residence time helps explain the delay between the actual sudden change in flow and the change observed at the flow sensor. Ignoring this delay can lead to misinterpretation, of incorrect timing of control action or confusion between a fault and, transport effect.

temporary

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

A most likely normal operation

B most possible explation is valve restriction

C Difficult to distinguish between pump failure hydraulic blockage and sensor failure

D Most likely normal operation, however there is no way of foreigning a fault

E Most likely unstable flow or instrumentation noise

- for additional information check pump operations, check hydraulic path, check sensor signal and use an independent flow check by measuring the collected water volume over time.

HARDWARE LIST

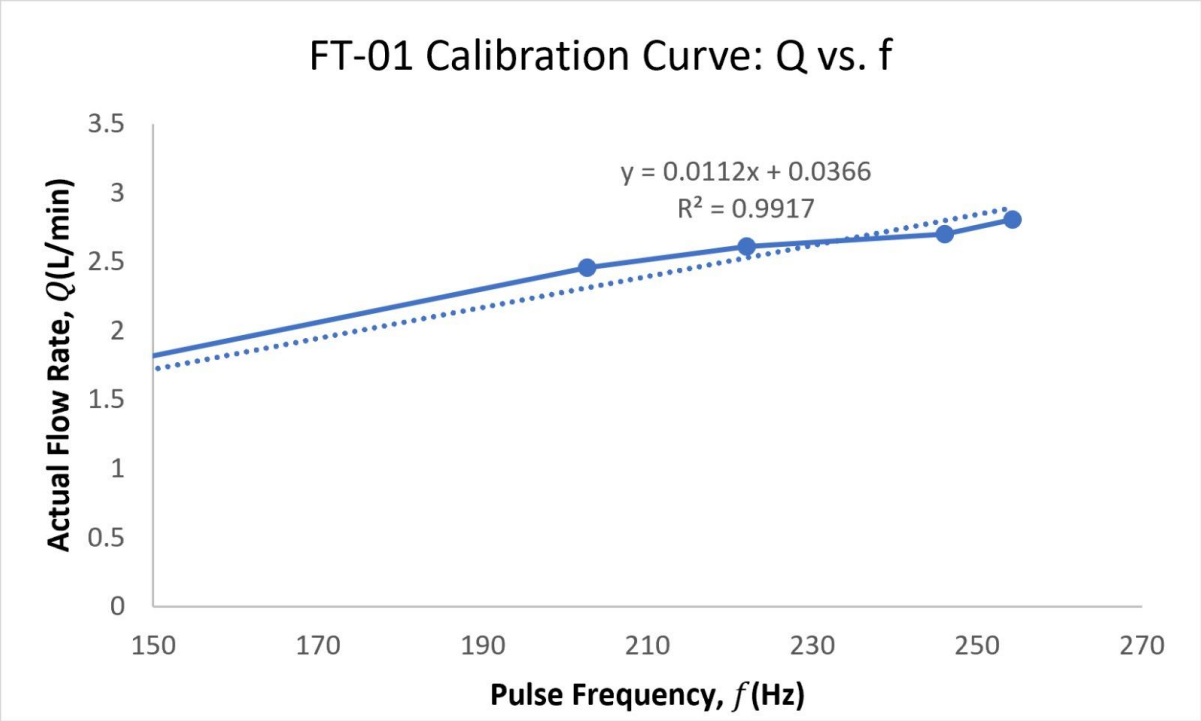
Mechanical Hardware

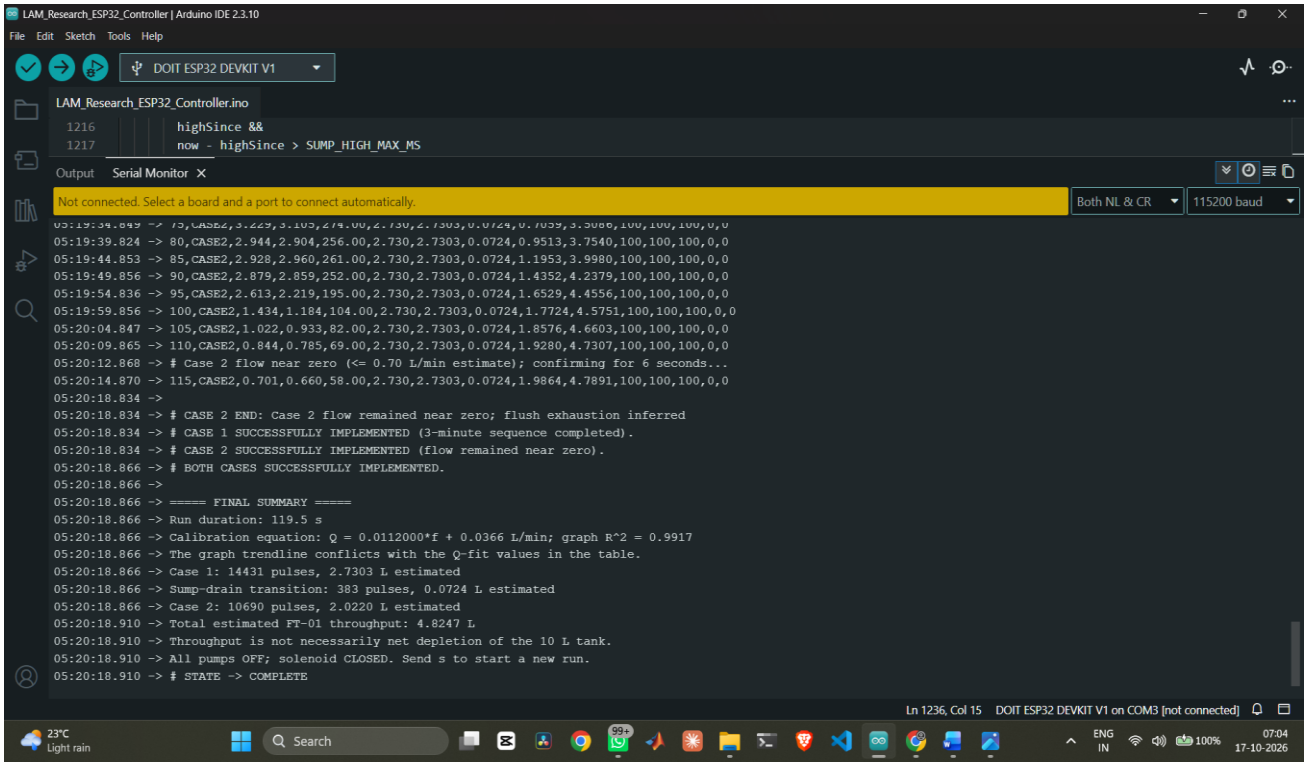
Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe**. Mainly used for Pumps
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line**.

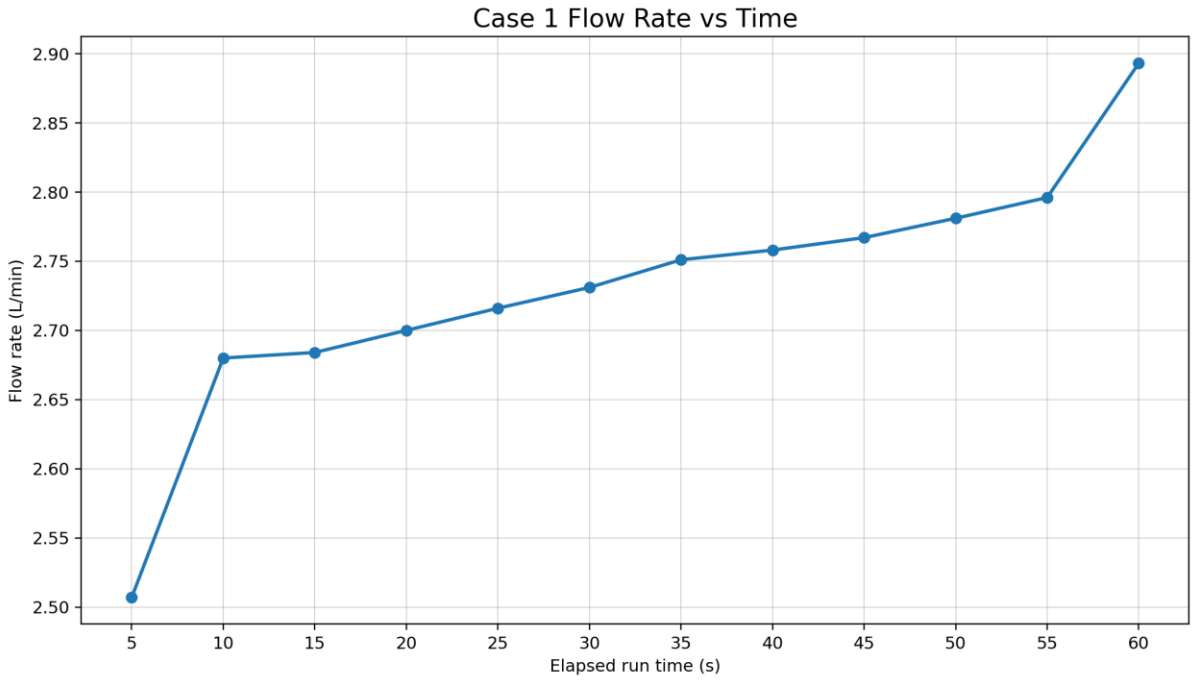
Electronic Hardware

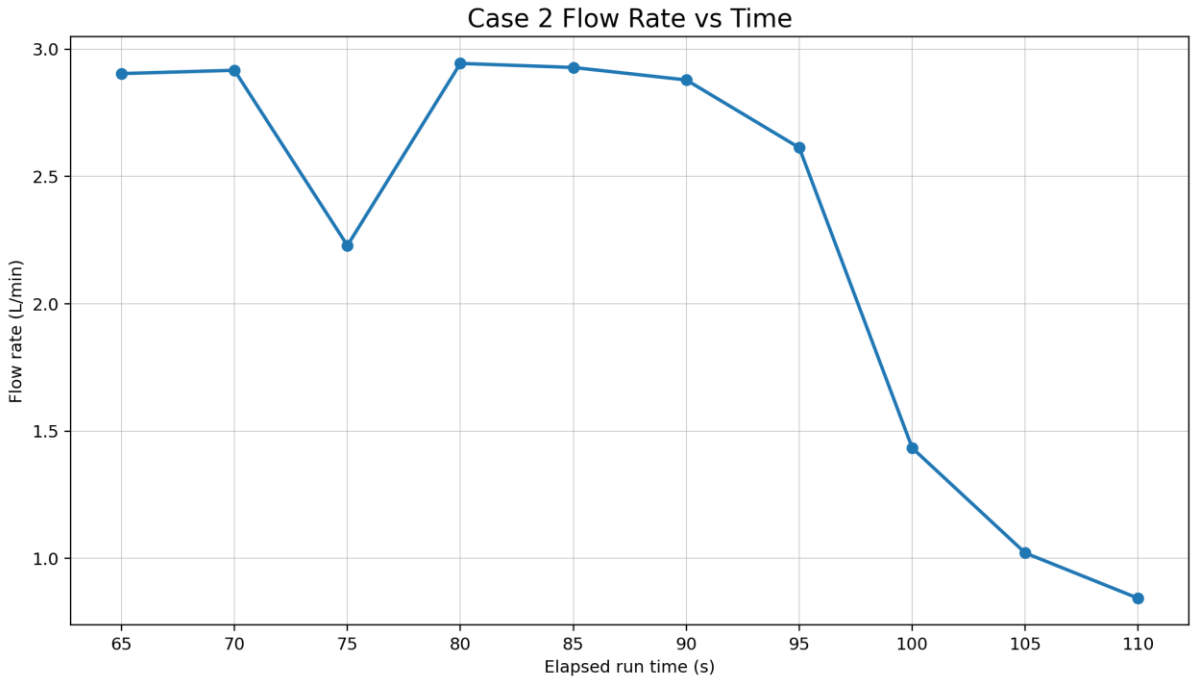
Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces**.
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller**. It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections**.
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board


FT-01 Calibration Curve: Q vs. f
y = 0.0112x + 0.0366
R² = 0.9917
Actual Flow Rate, Q(L/min)
Pulse Frequency, f(Hz)
3.5
3
2.5
2
1.5
1
0.5
0
150
170
190
210
230
250
270


LAM_Research_ESP32_Controller | Arduino IDE 2.3.10
File Edit Sketch Tools Help
DOIT ESP32 DEVKIT V1
LAM_Research_ESP32_Controller.ino
highSince &&
now - highSince > SUMP_HIGH_MAX_MS
Output Serial Monitor
Not connected. Select a board and a port to connect automatically.
Both NL & CR
115200 baud
05:19:39.824 -> 80,CASE2,2.944,2.904,256.00,2.730,2.7303,0.0724,0.9513,3.7540,100,100,100,0,0
05:19:44.853 -> 85,CASE2,2.928,2.960,261.00,2.730,2.7303,0.0724,1.1953,3.9980,100,100,100,0,0
05:19:49.856 -> 90,CASE2,2.879,2.859,252.00,2.730,2.7303,0.0724,1.4352,4.2379,100,100,100,0,0
05:19:54.836 -> 95,CASE2,2.613,2.219,195.00,2.730,2.7303,0.0724,1.6529,4.4556,100,100,100,0,0
05:19:59.856 -> 100,CASE2,1.434,1.184,104.00,2.730,2.7303,0.0724,1.7724,4.5751,100,100,100,0,0
05:20:04.847 -> 105,CASE2,1.022,0.933,82.00,2.730,2.7303,0.0724,1.8576,4.6603,100,100,100,0,0
05:20:09.865 -> 110,CASE2,0.844,0.785,69.00,2.730,2.7303,0.0724,1.9280,4.7307,100,100,100,0,0
05:20:12.868 -> # Case 2 flow near zero (<= 0.70 L/min estimate); confirming for 6 seconds...
05:20:14.870 -> 115,CASE2,0.701,0.660,58.00,2.730,2.7303,0.0724,1.9864,4.7891,100,100,100,0,0
05:20:18.834 ->
05:20:18.834 -> # CASE 2 END: Case 2 flow remained near zero; flush exhaustion inferred
05:20:18.834 -> # CASE 1 SUCCESSFULLY IMPLEMENTED (3-minute sequence completed).
05:20:18.834 -> # CASE 2 SUCCESSFULLY IMPLEMENTED (flow remained near zero).
05:20:18.866 -> # BOTH CASES SUCCESSFULLY IMPLEMENTED.
05:20:18.866 ->
05:20:18.866 -> ===== FINAL SUMMARY =====
05:20:18.866 -> Run duration: 119.5 s
05:20:18.866 -> Calibration equation: Q = 0.0112000*f + 0.0366 L/min; graph R^2 = 0.9917
05:20:18.866 -> The graph trendline conflicts with the Q-fit values in the table.
05:20:18.866 -> Case 1: 14431 pulses, 2.7303 L estimated
05:20:18.866 -> Sump-drain transition: 383 pulses, 0.0724 L estimated
05:20:18.866 -> Case 2: 10690 pulses, 2.0220 L estimated
05:20:18.910 -> Total estimated FT-01 throughput: 4.8247 L
05:20:18.910 -> Throughput is not necessarily net depletion of the 10 L tank.
05:20:18.910 -> All pumps OFF; solenoid CLOSED. Send s to start a new run.
05:20:18.910 -> # STATE -> COMPLETE
Ln 1236, Col 15 DOIT ESP32 DEVKIT V1 on COM3 [not connected]
23°C Light rain
Search
ENG IN
100%
07:04 17-10-2026


Case 1 Flow Rate vs Time
Flow rate (L/min)
2.90
2.85
2.80
2.75
2.70
2.65
2.60
2.55
2.50
5
10
15
20
25
30
35
40
45
50
55
60
Elapsed run time (s)


Case 2 Flow Rate vs Time
Flow rate (L/min)
3.0
2.5
2.0
1.5
1.0
65
70
75
80
85
90
95
100
105
110
Elapsed run time (s)

FINAL CODE

```
/*
 LAM'26  – STAGE 2 – TEAM SROTAS PRAVAHA

 Serial commands (Serial Monitor: 115200 baud, Newline):
   s       Start Case 1 -> sump-drain transition -> Case 2
   e       Emergency software stop: all pumps OFF, solenoid CLOSED
   r       Reset a FAULT state after checking the cause
   t1/t2/t3 Run a selected pump at 50% for 3 seconds (IDLE only)
   v       Open solenoid for 3 seconds (IDLE only)
   summary Print accumulated flow summary while stopped
   ?       Help

 Case 1:
   P1 = 50%, P2 = 100%, P3 = OFF, solenoid OPEN
   Sump-high interlock stops P1; P2 continues.
   Runs for 3 minutes. Flow is printed every 5 seconds.
   At Case 1 end, print average flow and estimated volume.

 Transition:
   Solenoid CLOSED, P1/P3 OFF, P2 = 100% to drain the sump.
   Case 2 starts when the float indicates level recovery.

 Case 2:
   P1 = 100%, P2 = 100%, P3 = 100% when the level interlock permits.
   Solenoid CLOSED throughout Case 2.
   Stops when near-zero flow persists after flow has been established,
   or at the 6-minute safety maximum.

 Calibration equation taken from the latest graph:
   Q [L/min] = 0.0112 * f [Hz] + 0.0366
   Graph-reported R^2 = 0.9917

 ALso:
   The graph equation conflicts with the Q-fit values in the supplied
   calibration table. This code deliberately uses the graph equation.
```

```
   Low-frequency values are extended approximately toward zero so that
   no pulses means exactly zero flow.
*/

#include <Arduino.h>
#include <ctype.h>
#include <string.h>

#if __has_include(<esp_arduino_version.h>)
 #include <esp_arduino_version.h>
#endif

#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
 #define LEDC_V3 1
#else
 #define LEDC_V3 0
#endif

// ============================== GPIO MAP ==============================

const uint8_t PIN_SW = 34;
const uint8_t PIN_FLOAT = 35;
const uint8_t PIN_FLOW = 14;
const uint8_t PIN_SV = 27;

const uint8_t P1_R_PWM = 33;
const uint8_t P1_L_PWM = 32;
const uint8_t P1_R_EN  = 26;
const uint8_t P1_L_EN  = 13;

const uint8_t P2_R_PWM = 19;
const uint8_t P2_L_PWM = 18;
const uint8_t P2_R_EN  = 17;
const uint8_t P2_L_EN  = 16;

const uint8_t P3_R_PWM = 25;
const uint8_t P3_L_PWM = 23;
const uint8_t P3_R_EN  = 22;
```

```
const uint8_t P3_L_EN  = 21;

// ============================ CONFIGURATION ============================

const int FLOAT_HIGH_LEVEL = HIGH;
const int SOLENOID_OPEN_LEVEL = HIGH;
const int SWITCH_ON_LEVEL = LOW;  // Diagnostic display only

const bool PUMP1_FWD_R = true;
const bool PUMP2_FWD_R = true;
const bool PUMP3_FWD_R = true;

// Case 1 pump duties
const uint8_t PUMP1_C1_PCT = 50;
const uint8_t PUMP2_C1_PCT = 100; //35.714 for 1L/min pumping

// Case 2 pump duties
const uint8_t PUMP1_C2_PCT = 100;
const uint8_t PUMP2_C2_PCT = 100;
const uint8_t PUMP3_C2_PCT = 100;

// Corrected Case 1 time: 60000 ms = 1 minute
const uint32_t CASE1_DURATION_MS = 60000UL;

// Case 2 safety maximum: 6 minutes
const uint32_t CASE2_MAX_DURATION_MS = 360000UL;

const uint32_t CASE2_MIN_RUN_MS = 30000UL;
const uint32_t CASE2_NEAR_ZERO_HOLD_MS = 6000UL;
const float CASE2_NEAR_ZERO_FLOW_LPM = 0.70f;

const uint32_t CASE2_PRIME_TIMEOUT_MS = 25000UL;
const uint32_t FLOW_ESTABLISH_PULSES = 10UL;

const uint32_t SOLENOID_SETTLE_MS = 1500UL;
const uint32_t TRANSITION_TIMEOUT_MS = 60000UL;
const uint32_t LEVEL_RECOVER_DELAY_MS = 3000UL;
const uint32_t SUMP_HIGH_MAX_MS = 120000UL;
```

```
const uint32_t FLOAT_DEBOUNCE_MS = 500UL;

// Internal flow sampling every second; telemetry every 5 seconds
const uint32_t FLOW_SAMPLE_MS = 1000UL;
const uint32_t TELEMETRY_MS = 5000UL;

// ============================== CALIBRATION ==============================

// Equation shown in the calibration graph:
// Q [L/min] = CAL_A * f [Hz] + CAL_B
const float CAL_A = 0.0112f;
const float CAL_B = 0.0366f;
const float CAL_R_SQUARED = 0.9917f;

// Frequency range covered by the four supplied trials
const float CAL_FREQ_MIN_HZ = 202.65f;
const float CAL_FREQ_MAX_HZ = 254.30f;

// PWM
const uint32_t PWM_FREQ_HZ = 20000UL;
const uint8_t PWM_BITS = 8;

// Diagnostic pump tests
const uint8_t TEST_PCT = 50;
const uint32_t TEST_MS = 3000UL;

// ============================== STATE AND DATA ==============================

enum State : uint8_t {
  ST_IDLE,
  ST_CASE1,
  ST_TRANSITION,
  ST_CASE2,
  ST_COMPLETE,
  ST_FAULT,
  ST_ESTOP
};
```

```
struct PumpDef {
  uint8_t rPwm;
  uint8_t lPwm;
  uint8_t rEn;
  uint8_t lEn;
  uint8_t rCh;
  uint8_t lCh;
  bool fwdR;
  uint8_t pct;
};

PumpDef pumps[3] = {
  {
    P1_R_PWM, P1_L_PWM, P1_R_EN, P1_L_EN,
    0, 1, PUMP1_FWD_R, 0
  },
  {
    P2_R_PWM, P2_L_PWM, P2_R_EN, P2_L_EN,
    2, 3, PUMP2_FWD_R, 0
  },
  {
    P3_R_PWM, P3_L_PWM, P3_R_EN, P3_L_EN,
    4, 5, PUMP3_FWD_R, 0
  }
};

State state = ST_IDLE;

uint32_t stateStart = 0;
uint32_t runStart = 0;
bool runStarted = false;

bool solOpen = false;
bool sumpHigh = false;
bool levelHold = false;

uint32_t lastHighMs = 0;
uint32_t highSince = 0;
```

```
bool dbFlStable = false;
bool dbFlLast = false;
uint32_t dbFlChangedAt = 0;

volatile uint32_t pulseCount = 0;

uint32_t lastSampleMs = 0;
uint32_t lastSamplePulses = 0;
uint32_t lastPulseMs = 0;

float sF = 0.0f;
float sQ = 0.0f;
bool flowSampleUpdated = false;

uint32_t pulses1 = 0;
uint32_t pulsesTransition = 0;
uint32_t pulses2 = 0;

double vol1 = 0.0;
double volTransition = 0.0;
double vol2 = 0.0;

uint32_t lastTelemetryMs = 0;
uint32_t lastTelemetryPulses = 0;
uint32_t lastIdlePrintMs = 0;

bool nearZeroTiming = false;
uint32_t nearZeroSince = 0;

bool case2FlowEstablished = false;

int testIdx = -1;
uint32_t testUntil = 0;

char cmdBuf[24];
uint8_t cmdLen = 0;
```

```
// ============================ STATE NAME ============================

const char* stateName(State s) {
 switch (s) {
  case ST_IDLE:      return "IDLE";
  case ST_CASE1:     return "CASE1";
  case ST_TRANSITION: return "TRANSITION";
  case ST_CASE2:     return "CASE2";
  case ST_COMPLETE:   return "COMPLETE";
  case ST_FAULT:     return "FAULT";
  case ST_ESTOP:     return "E_STOP";
 }

 return "?";
}

// ============================ PWM HELPERS ============================

void pwmAttach(uint8_t pin, uint8_t channel) {
#if LEDC_V3
 (void)channel;
 ledcAttach(pin, PWM_FREQ_HZ, PWM_BITS);
#else
 ledcSetup(channel, PWM_FREQ_HZ, PWM_BITS);
 ledcAttachPin(pin, channel);
#endif
}

void pwmWrite(uint8_t pin, uint8_t channel, uint32_t duty) {
#if LEDC_V3
 (void)channel;
 ledcWrite(pin, duty);
#else
 (void)pin;
 ledcWrite(channel, duty);
#endif
}
```

```
// ============================ PUMP CONTROL ============================

void setPump(int i, uint8_t pct) {
  PumpDef &p = pumps[i];
  p.pct = pct;

  if (pct == 0) {
    pwmWrite(p.rPwm, p.rCh, 0);
    pwmWrite(p.lPwm, p.lCh, 0);

    digitalWrite(p.rEn, LOW);
    digitalWrite(p.lEn, LOW);
    return;
  }

  uint32_t duty = (pct >= 100)
                  ? 255U
                  : ((uint32_t)pct * 255U + 50U) / 100U;

  digitalWrite(p.rEn, HIGH);
  digitalWrite(p.lEn, HIGH);

  if (p.fwdR) {
    pwmWrite(p.rPwm, p.rCh, duty);
    pwmWrite(p.lPwm, p.lCh, 0);
  } else {
    pwmWrite(p.rPwm, p.rCh, 0);
    pwmWrite(p.lPwm, p.lCh, duty);
  }
}

void setSolenoid(bool open) {
  solOpen = open;

  digitalWrite(
    PIN_SV,
    open
      ? SOLENOID_OPEN_LEVEL
```

```
    : (SOLENOID_OPEN_LEVEL == HIGH ? LOW : HIGH)
  );
}

void stopAll() {
  for (int i = 0; i < 3; i++) {
    setPump(i, 0);
  }

  setSolenoid(false);
}

// ============================ FLOW SENSOR ============================

void IRAM_ATTR flowISR() {
  pulseCount++;
}

uint32_t pulsesNow() {
  noInterrupts();
  uint32_t value = pulseCount;
  interrupts();

  return value;
}

// Convert pulse frequency to flow.
// The low-frequency extension prevents the fitted positive intercept
// from falsely reporting substantial flow at very low frequencies.
// No pulses in the sample interval means exactly 0 L/min.
float calibratedFlowLpm(float frequencyHz, uint32_t deltaPulses) {
  if (deltaPulses == 0 || frequencyHz <= 0.0f) {
    return 0.0f;
  }

  float q;

  if (frequencyHz < CAL_FREQ_MIN_HZ) {
```

```
    const float qAtMin =
      CAL_A * CAL_FREQ_MIN_HZ + CAL_B;

    q = qAtMin * (frequencyHz / CAL_FREQ_MIN_HZ);
  } else {
    q = CAL_A * frequencyHz + CAL_B;
  }

  return (q > 0.0f) ? q : 0.0f;
}

// Sample flow and accumulate estimated volume for the active phase.
void sampleFlow(uint32_t now, bool force = false) {
  if (!force && now - lastSampleMs < FLOW_SAMPLE_MS) {
    return;
  }

  uint32_t totalPulses = pulsesNow();
  uint32_t deltaPulses = totalPulses - lastSamplePulses;

  float dt = (now - lastSampleMs) / 1000.0f;

  if (dt <= 0.0f) {
    return;
  }

  lastSampleMs = now;
  lastSamplePulses = totalPulses;

  sF = deltaPulses / dt;
  sQ = calibratedFlowLpm(sF, deltaPulses);

  if (deltaPulses > 0) {
    lastPulseMs = now;
  }

  const double intervalVolumeL =
    (double)sQ * (double)dt / 60.0;
```

```
  if (state == ST_CASE1) {
    pulses1 += deltaPulses;
    vol1 += intervalVolumeL;
  } else if (state == ST_TRANSITION) {
    pulsesTransition += deltaPulses;
    volTransition += intervalVolumeL;
  } else if (state == ST_CASE2) {
    pulses2 += deltaPulses;
    vol2 += intervalVolumeL;

    if (pulses2 >= FLOW_ESTABLISH_PULSES) {
      case2FlowEstablished = true;
    }
  }

  flowSampleUpdated = true;
}

// ============================== FLOAT SWITCH ==============================

void updateFloatInput(uint32_t now) {
  bool rawActive =
    (digitalRead(PIN_FLOAT) == FLOAT_HIGH_LEVEL);

  if (rawActive != dbFlLast) {
    dbFlLast = rawActive;
    dbFlChangedAt = now;
  }

  if (now - dbFlChangedAt >= FLOAT_DEBOUNCE_MS) {
    dbFlStable = dbFlLast;
  }

  sumpHigh = dbFlStable;
}

void updateLevel(uint32_t now) {
```

```
  if (sumpHigh) {
    if (!levelHold) {
      levelHold = true;
      highSince = now;

      Serial.println(
        "# SUMP HIGH: Pump 1 and Pump 3 OFF; Pump 2 continues"
      );
    }

    lastHighMs = now;
  } else if (
    levelHold &&
    now - lastHighMs >= LEVEL_RECOVER_DELAY_MS
  ) {
    levelHold = false;
    highSince = 0;

    Serial.println("# SUMP level recovered; interlock cleared");
  }
}

// ============================== REPORTING ==============================

void printHeader() {
  Serial.println(
    "t_s,state,flow_5s_avg_Lmin,flow_last_1s_Lmin,"
    "frequency_Hz,case1_avg_Lmin,case1_L_est,"
    "transition_L_est,case2_L_est,total_L_est,"
    "P1_pct,P2_pct,P3_pct,SV_open,sumpHigh"
  );
}

void printTelemetry(uint32_t now) {
  uint32_t totalPulses = pulsesNow();
  uint32_t pulseDelta = totalPulses - lastTelemetryPulses;

  float dt = (now - lastTelemetryMs) / 1000.0f;
```

```
if (dt <= 0.0f) {
  dt = 0.001f;
}

const float freq5 = pulseDelta / dt;
const float q5 = calibratedFlowLpm(freq5, pulseDelta);

float case1ElapsedMin;

if (state == ST_CASE1) {
  case1ElapsedMin = (now - stateStart) / 60000.0f;
} else {
  case1ElapsedMin = CASE1_DURATION_MS / 60000.0f;
}

float case1Avg =
  (case1ElapsedMin > 0.0f)
    ? (float)(vol1 / case1ElapsedMin)
    : 0.0f;

Serial.printf(
  "%lu,%s,%.3f,%.3f,%.2f,%.3f,%.4f,%.4f,%.4f,%.4f,"
  "%u,%u,%u,%d,%d\n",
  (unsigned long)((now - runStart) / 1000UL),
  stateName(state),
  q5,
  sQ,
  sF,
  case1Avg,
  (float)vol1,
  (float)volTransition,
  (float)vol2,
  (float)(vol1 + volTransition + vol2),
  (unsigned)pumps[0].pct,
  (unsigned)pumps[1].pct,
  (unsigned)pumps[2].pct,
  solOpen ? 1 : 0,
```

```
    sumpHigh ? 1 : 0
  );

  lastTelemetryMs = now;
  lastTelemetryPulses = totalPulses;
}

void printCase1EndReport(uint32_t now) {
  sampleFlow(now, true);

  const double minutes =
    (now - stateStart) / 60000.0;

  const double averageLpm =
    (minutes > 0.0) ? vol1 / minutes : 0.0;

  Serial.println();
  Serial.println(
    "========== CASE 1 FLOW REPORT (BEFORE CASE 2) =========="
  );

  Serial.printf(
    "Case 1 elapsed time: %.1f s\n",
    (now - stateStart) / 1000.0f
  );

  Serial.printf(
    "Pulses counted during Case 1: %lu\n",
    (unsigned long)pulses1
  );

  Serial.printf(
    "Estimated average return flow: %.3f L/min\n",
    (float)averageLpm
  );

  Serial.printf(
    "Estimated water returned toward the 10 L tank: %.4f L\n",
```

```
    (float)vol1
  );

  Serial.println(
    "Volume is integrated from flow samples: sum(Q * dt / 60)."
  );

  Serial.printf(
    "Calibration used: Q = %.7f*f + %.4f; graph R^2 = %.4f\n",
    CAL_A, CAL_B, CAL_R_SQUARED
  );

  Serial.printf(
    "Frequency range of supplied trials: %.2f-%.2f Hz\n",
    CAL_FREQ_MIN_HZ, CAL_FREQ_MAX_HZ
  );

  Serial.println(
    "WARNING: graph equation conflicts with Q-fit values in the table."
  );

  Serial.println(
    "Estimates outside the calibration range are extrapolations."
  );

  Serial.println(
    "============================================================"
  );
}

void printTransitionReport() {
  Serial.println();
  Serial.println(
    "========== SUMP-DRAIN TRANSITION REPORT =========="
  );

  Serial.printf(
    "Additional estimated volume during sump drain: %.4f L\n",
```

```
    (float)volTransition
  );

  Serial.printf(
    "Cumulative estimate through Case 1 + transition: %.4f L\n",
    (float)(vol1 + volTransition)
  );

  Serial.println("Case 2 is starting next.");
  Serial.println("Solenoid valve is commanded CLOSED.");
  Serial.println(
    "================================================="
  );
}

void printSummary(const char* title) {
  Serial.printf("\n===== %s =====\n", title);

  if (runStarted) {
    Serial.printf(
      "Run duration: %.1f s\n",
      (millis() - runStart) / 1000.0f
    );
  }

  Serial.printf(
    "Calibration equation: Q = %.7f*f + %.4f L/min; graph R^2 = %.4f\n",
    CAL_A, CAL_B, CAL_R_SQUARED
  );

  Serial.println(
    "The graph trendline conflicts with the Q-fit values in the table."
  );

  Serial.printf(
    "Case 1: %lu pulses, %.4f L estimated\n",
    (unsigned long)pulses1, (float)vol1
  );
```

```
  Serial.printf(
    "Sump-drain transition: %lu pulses, %.4f L estimated\n",
    (unsigned long)pulsesTransition, (float)volTransition
  );

  Serial.printf(
    "Case 2: %lu pulses, %.4f L estimated\n",
    (unsigned long)pulses2, (float)vol2
  );

  Serial.printf(
    "Total estimated FT-01 throughput: %.4f L\n",
    (float)(vol1 + volTransition + vol2)
  );

  Serial.println(
    "Throughput is not necessarily net depletion of the 10 L tank."
  );
}

void gotoState(State next, uint32_t now) {
  state = next;
  stateStart = now;

  Serial.printf(
    "# STATE -> %s\n",
    stateName(state)
  );
}

void enterFault(const char* message, uint32_t now) {
  stopAll();
  nearZeroTiming = false;

  Serial.printf(
    "\n!!! FAULT: %s\n",
    message
```

```
  );

  printSummary("FAULT SUMMARY");

  Serial.println(
    "All pumps OFF; solenoid CLOSED. Inspect the cause, then send r."
  );

  gotoState(ST_FAULT, now);
}

void finishComplete(
  const char* reason,
  uint32_t now,
  bool flushExhaustionConfirmed
) {
  sampleFlow(now, true);
  stopAll();
  nearZeroTiming = false;

  Serial.printf(
    "\n# CASE 2 END: %s\n",
    reason
  );

  if (flushExhaustionConfirmed) {
    Serial.println(
      "# CASE 1 SUCCESSFULLY IMPLEMENTED (3-minute sequence completed)."
    );

    Serial.println(
      "# CASE 2 SUCCESSFULLY IMPLEMENTED (flow remained near zero)."
    );

    Serial.println(
      "# BOTH CASES SUCCESSFULLY IMPLEMENTED."
    );
  } else {
```

```
    Serial.println(
      "# Case 2 stopped at the 6-minute safety limit."
    );

    Serial.println(
      "# Flush exhaustion was NOT confirmed by the flow criterion."
    );
  }

  printSummary("FINAL SUMMARY");

  Serial.println(
    "All pumps OFF; solenoid CLOSED. Send s to start a new run."
  );

  gotoState(ST_COMPLETE, now);
}

void emergencyStop(uint32_t now) {
  bool wasRunning =
    (state == ST_CASE1 ||
     state == ST_TRANSITION ||
     state == ST_CASE2);

  stopAll();

  testIdx = -1;
  nearZeroTiming = false;

  if (wasRunning) {
    printSummary("EMERGENCY STOP SUMMARY");
  }

  Serial.println(
    "\n!!! EMERGENCY STOP: all pumps OFF, solenoid CLOSED !!!"
  );

  gotoState(ST_ESTOP, now);
```

```
  Serial.println(
    "No automatic restart. Send s only after checking the system."
  );
}

// ============================== CASE 1 PUMP CONTROL ==============================

void applyCase1Pumps() {
  setSolenoid(true);

  setPump(
    0,
    levelHold ? 0 : PUMP1_C1_PCT
  );

  setPump(1, PUMP2_C1_PCT);
  setPump(2, 0);
}

// ============================== START THE FULL RUN ==============================

void startRun(uint32_t now) {
  stopAll();

  testIdx = -1;

  runStart = now;
  runStarted = true;

  pulses1 = 0;
  pulsesTransition = 0;
  pulses2 = 0;

  vol1 = 0.0;
  volTransition = 0.0;
  vol2 = 0.0;
```

```
  levelHold = sumpHigh;
  lastHighMs = now;
  highSince = sumpHigh ? now : 0;

  lastPulseMs = now;
  nearZeroTiming = false;
  case2FlowEstablished = false;

  lastSampleMs = now;
  lastSamplePulses = pulsesNow();

  sF = 0.0f;
  sQ = 0.0f;
  flowSampleUpdated = false;

  lastTelemetryMs = now;
  lastTelemetryPulses = pulsesNow();

  gotoState(ST_CASE1, now);

  printHeader();

  Serial.println(
    "# RUN STARTED by serial command s."
  );

  Serial.println(
    "# Confirm manual flush ball valve is OPEN before Case 2."
  );

  Serial.println(
    "# Case 1: P1=50%, P2=100%, P3=OFF, solenoid OPEN."
  );

  applyCase1Pumps();
}

// ============================ SERIAL COMMANDS ============================
```

```
void processCommand(char* text, uint32_t now) {
 while (*text == ' ' || *text == '\t') {
  text++;
 }

 if (*text == '\0') {
  return;
 }

 char c = (char)tolower((unsigned char)text[0]);

 // Emergency stop is accepted in every state.
 if (c == 'e' || c == 'x') {
  emergencyStop(now);
  return;
 }

 if (strcmp(text, "summary") == 0) {
  if (state == ST_CASE1 ||
      state == ST_TRANSITION ||
      state == ST_CASE2) {
    Serial.println(
      "# Run active; summary will be printed when the run stops."
    );
  } else {
    printSummary("FLOW SUMMARY");
  }

  return;
 }

 if (strcmp(text, "?") == 0 || c == 'h') {
  Serial.println(
    "Commands: s=start | e=emergency stop | r=reset fault | summary"
  );

  Serial.println(
```

```
    "Diagnostics in IDLE: t1/t2/t3=pump test, v=solenoid test"
  );

  return;
 }

 if (c == 's' &&
     (text[1] == '\0' ||
      text[1] == ' ' ||
      text[1] == '\t')) {

  if (state == ST_CASE1 ||
      state == ST_TRANSITION ||
      state == ST_CASE2) {
    Serial.println(
      "# Sequence is already running; send e to stop."
    );
    return;
  }

  if (state == ST_FAULT) {
    Serial.println(
      "# FAULT is latched. Inspect the cause, send r, then s."
    );
    return;
  }

  if (testIdx >= 0) {
    Serial.println(
      "# Wait for diagnostic test to finish, or send e."
    );
    return;
  }

  startRun(now);
  return;
 }
```

```
if (c == 'r') {
  if (state == ST_FAULT) {
    stopAll();

    testIdx = -1;
    gotoState(ST_IDLE, now);

    Serial.println(
      "# FAULT reset. Send s after confirming the cause is resolved."
    );
  } else {
    Serial.println(
      "# r resets FAULT only. COMPLETE/E_STOP can be restarted with s."
    );
  }

  return;
}

if (state != ST_IDLE) {
  Serial.println(
    "# Diagnostic tests require IDLE. Send e to stop or s to start."
  );

  return;
}

if (c == 't' &&
    text[1] >= '1' &&
    text[1] <= '3') {
  testIdx = text[1] - '1';
} else if (c == 'v') {
  testIdx = 3;
} else {
  Serial.println(
    "# Commands: s, e, r, t1, t2, t3, v, summary, ?"
  );
  return;
```

```
  }

  testUntil = now + TEST_MS;

  Serial.println(
    "# Diagnostic test running for 3 seconds."
  );
}

void handleSerial(uint32_t now) {
  while (Serial.available() > 0) {
    char ch = (char)Serial.read();

    if (ch == '\r') {
      continue;
    }

    if (ch == '\n') {
      cmdBuf[cmdLen] = '\0';

      if (cmdLen > 0) {
        processCommand(cmdBuf, now);
      }

      cmdLen = 0;
    } else if (cmdLen < sizeof(cmdBuf) - 1) {
      cmdBuf[cmdLen++] = ch;
    } else {
      cmdLen = 0;

      Serial.println(
        "# Input too long; command cleared."
      );
    }
  }
}

// ============================ SETUP ============================
```

```
void setup() {
  const uint8_t outputs[] = {
    P1_R_PWM, P1_L_PWM, P1_R_EN, P1_L_EN,
    P2_R_PWM, P2_L_PWM, P2_R_EN, P2_L_EN,
    P3_R_PWM, P3_L_PWM, P3_R_EN, P3_L_EN
  };

  for (
    uint8_t i = 0;
    i < sizeof(outputs) / sizeof(outputs[0]);
    i++
  ) {
    pinMode(outputs[i], OUTPUT);
    digitalWrite(outputs[i], LOW);
  }

  pinMode(PIN_SV, OUTPUT);
  setSolenoid(false);

  for (int i = 0; i < 3; i++) {
    pwmAttach(pumps[i].rPwm, pumps[i].rCh);
    pwmAttach(pumps[i].lPwm, pumps[i].lCh);
  }

  stopAll();

  pinMode(PIN_SW, INPUT);
  pinMode(PIN_FLOAT, INPUT);
  pinMode(PIN_FLOW, INPUT);

  Serial.begin(115200);
  delay(300);

  uint32_t now = millis();

  dbFlLast = dbFlStable =
    (digitalRead(PIN_FLOAT) == FLOAT_HIGH_LEVEL);
```

```
dbFlChangedAt = now;
sumpHigh = dbFlStable;

lastSampleMs = now;
lastSamplePulses = pulsesNow();

lastTelemetryMs = now;
lastTelemetryPulses = pulsesNow();

attachInterrupt(
 digitalPinToInterrupt(PIN_FLOW),
 flowISR,
 FALLING
);

Serial.println(
 "\nLam Research Challenge 3.0 - Stage 2 controller"
);

Serial.println(
 "Startup safe state: all pumps OFF; solenoid CLOSED."
);

Serial.println(
 "Send s to start Case 1 -> transition -> Case 2."
);

Serial.println(
 "Send e at any time for emergency software stop."
);

Serial.printf(
 "Calibration equation from latest graph: Q = %.4f*f + %.4f L/min; R^2=%.4f\n",
 CAL_A, CAL_B, CAL_R_SQUARED
);

Serial.printf(
```

```
    "Supplied trial frequency range: %.2f-%.2f Hz\n",
    CAL_FREQ_MIN_HZ, CAL_FREQ_MAX_HZ
  );

  Serial.println(
    "WARNING: graph trendline conflicts with the Q-fit table."
  );

  Serial.println(
    "Open the manual flush ball valve before sending s."
  );

  Serial.printf(
    "Initial switch raw=%d (diagnostic only); FLOAT raw=%d (HIGH means sump-high)\n",
    digitalRead(PIN_SW),
    digitalRead(PIN_FLOAT)
  );
}

// ============================== MAIN LOOP ==============================

void loop() {
  uint32_t now = millis();

  updateFloatInput(now);

  // Handle the emergency-stop command before outputs are applied again.
  handleSerial(now);

  sampleFlow(now, false);

  if (flowSampleUpdated) {
    flowSampleUpdated = false;

    if (
      (state == ST_CASE1 ||
       state == ST_TRANSITION ||
       state == ST_CASE2) &&
```

```
    now - lastTelemetryMs >= TELEMETRY_MS
  ) {
    printTelemetry(now);
  }
}

switch (state) {
  case ST_IDLE: {
    if (testIdx >= 0) {
      if (testIdx < 3) {
        setPump(testIdx, TEST_PCT);
      } else {
        setSolenoid(true);
      }

      if ((int32_t)(now - testUntil) >= 0) {
        stopAll();

        testIdx = -1;

        Serial.println(
          "# Diagnostic test complete; outputs OFF."
        );
      }
    } else if (now - lastIdlePrintMs >= 3000UL) {
      lastIdlePrintMs = now;

      Serial.printf(
        "# IDLE | send s to start | SW raw=%d | FLOAT raw=%d sumpHigh=%d | flow=%.3f L/min | pulses=%lu\n",
        digitalRead(PIN_SW),
        digitalRead(PIN_FLOAT),
        sumpHigh ? 1 : 0,
        sQ,
        (unsigned long)pulsesNow()
      );
    }

    break;
```

```
}

case ST_CASE1: {
  updateLevel(now);
  applyCase1Pumps();

  uint32_t elapsed = now - stateStart;

  if (
    SUMP_HIGH_MAX_MS &&
    highSince &&
    now - highSince > SUMP_HIGH_MAX_MS
  ) {
    enterFault(
      "Sump remained HIGH too long during Case 1",
      now
    );
    break;
  }

  if (elapsed >= CASE1_DURATION_MS) {
    printCase1EndReport(now);

    Serial.println(
      "# CASE 1 complete (3 minutes): solenoid CLOSED, P1/P3 OFF, P2 drains sump."
    );

    setSolenoid(false);
    setPump(0, 0);
    setPump(2, 0);
    setPump(1, PUMP2_C1_PCT);

    nearZeroTiming = false;

    gotoState(ST_TRANSITION, now);
  }

  break;
```

```
}

case ST_TRANSITION: {
  updateLevel(now);

  // Keep solenoid CLOSED throughout transition.
  setSolenoid(false);
  setPump(0, 0);
  setPump(2, 0);
  setPump(1, PUMP2_C2_PCT);

  uint32_t elapsed = now - stateStart;

  if (
    elapsed >= SOLENOID_SETTLE_MS &&
    !levelHold
  ) {
    // Attribute remaining pulses to the transition before changing state.
    sampleFlow(now, true);

    printTransitionReport();

    case2FlowEstablished = false;
    nearZeroTiming = false;

    // Explicitly close the solenoid as Case 2 begins.
    setSolenoid(false);
    gotoState(ST_CASE2, now);
    setSolenoid(false);

    Serial.println(
      "# SOLENOID CLOSED for Case 2."
    );

    Serial.println(
      "# CASE 2 START: P1=100%, P2=100%, P3=100%, subject to sump-high interlock."
    );
  } else if (
```

```
    elapsed >= TRANSITION_TIMEOUT_MS
   ) {
    enterFault(
     "Sump did not clear before Case 2; inspect float, Pump 2, and plumbing",
     now
    );
   }

   break;
  }

  case ST_CASE2: {
   updateLevel(now);

   // Keep the solenoid closed on every Case 2 loop.
   setSolenoid(false);

   // Safety interlock: if sump level is high, P1/P3 pause while P2 drains.
   const bool allowP1P3 = !levelHold;

   setPump(
    0,
    allowP1P3 ? PUMP1_C2_PCT : 0
   );

   setPump(1, PUMP2_C2_PCT);

   setPump(
    2,
    allowP1P3 ? PUMP3_C2_PCT : 0
   );

   uint32_t elapsed = now - stateStart;

   if (
    SUMP_HIGH_MAX_MS &&
    highSince &&
    now - highSince > SUMP_HIGH_MAX_MS
```

```
) {
  enterFault(
    "Sump remained HIGH too long in Case 2",
    now
  );
  break;
}

// Do not declare successful flush exhaustion if no flow ever established.
if (
  !levelHold &&
  !case2FlowEstablished &&
  elapsed >= CASE2_PRIME_TIMEOUT_MS
) {
  enterFault(
    "Case 2 flow did not establish; check flush supply, valve, pumps, and sensor",
    now
  );
  break;
}

// Detect flush exhaustion only after measured flow was established.
if (
  elapsed >= CASE2_MIN_RUN_MS &&
  case2FlowEstablished &&
  !levelHold
) {
  if (sQ <= CASE2_NEAR_ZERO_FLOW_LPM) {
    if (!nearZeroTiming) {
      nearZeroTiming = true;
      nearZeroSince = now;

      Serial.printf(
        "# Case 2 flow near zero (<= %.2f L/min estimate); confirming for %lu seconds...\n",
        CASE2_NEAR_ZERO_FLOW_LPM,
        (unsigned long)(CASE2_NEAR_ZERO_HOLD_MS / 1000UL)
      );
    } else if (
```

```
        now - nearZeroSince >= CASE2_NEAR_ZERO_HOLD_MS
      ) {
        finishComplete(
          "Case 2 flow remained near zero; flush exhaustion inferred",
          now,
          true
        );

        break;
      }
    } else {
      nearZeroTiming = false;
      nearZeroSince = 0;
    }
  } else {
    nearZeroTiming = false;
    nearZeroSince = 0;
  }

  // Safety maximum if the near-zero-flow condition is not met.
  if (elapsed >= CASE2_MAX_DURATION_MS) {
    finishComplete(
      "Case 2 six-minute safety maximum reached",
      now,
      false
    );

    break;
  }

  break;
}

case ST_COMPLETE:
case ST_ESTOP:
  // Latched safe-output states. A deliberate 's' starts a new run.
  break;
```

```
    case ST_FAULT:
      // Latched fault; inspect the cause and send 'r' to reset.
      break;
  }
}
```

GIT REPO OF THE FULL PROJECT :

https://github.com/uncomparableX/Srotas.Pravaha_LAM-26_Solution